Hashing and Hash Tables

[CSE2010] Data Structures

Department of Data Science

Introduction

- Program that allows customers to order fast food from a restaurant
  - Menu of foods with their respective prices

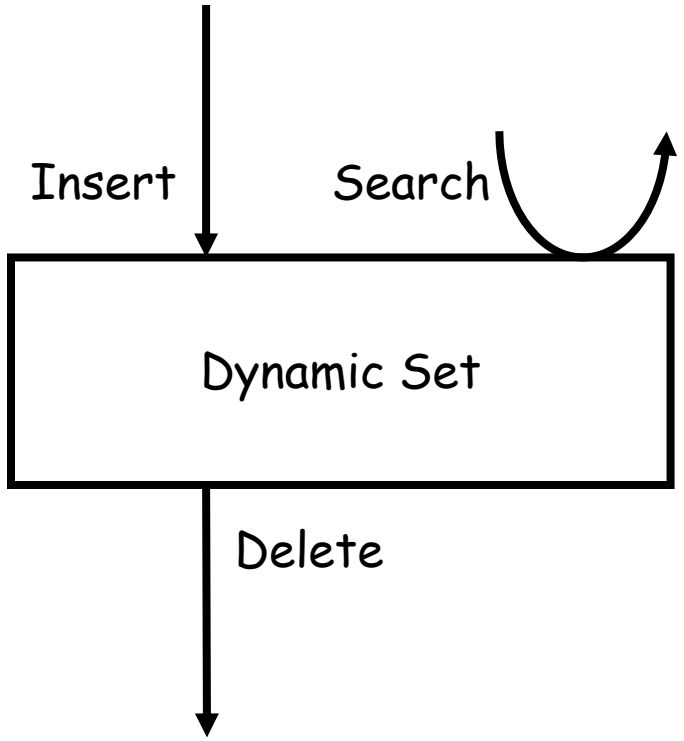
- Searching for the price of a given food
  - $O(n)$ time for linear search in the unsorted array
  - $O(logn)$ time for binary search in the sorted array
  - Can we do this better?

Menu

0	French Fries	0.75
1	Hamburger	2.5
2	Hot Dog	1.5
3	Soda	0.6

Dynamic Set

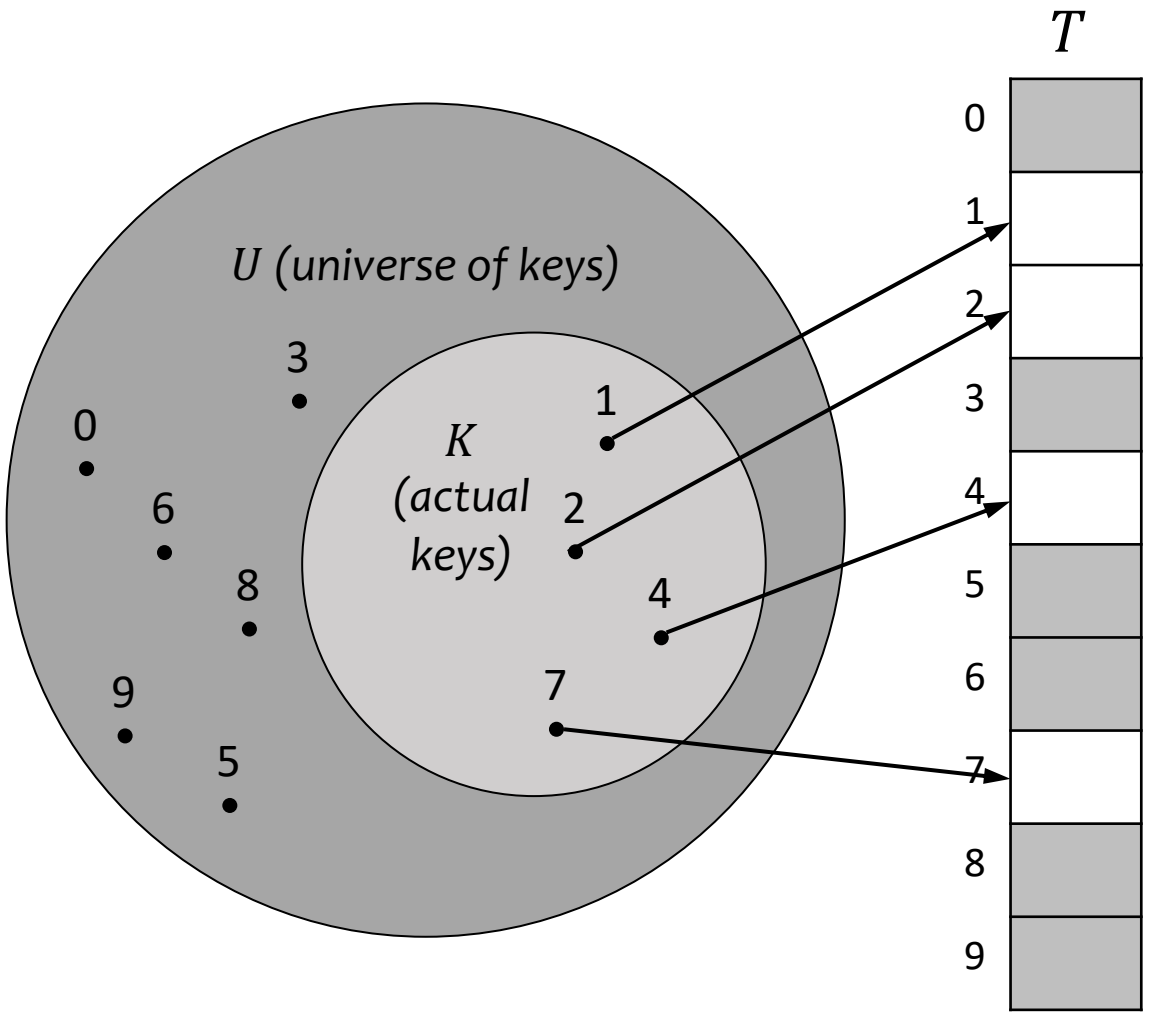
- Many applications requires a **dynamic set** that supports:
  - Inserting a key
  - Deleting a key
  - Searching for a key



Direct Addressing

- Simple technique working well when the universe U of keys is reasonably small.

- What if
  - U is so large that storing T of size |U| is impossible, given the memory available.
  - K is so small relative to U



Hashing & Hash Tables

Hashing is the process of taking elements and converting them to numbers

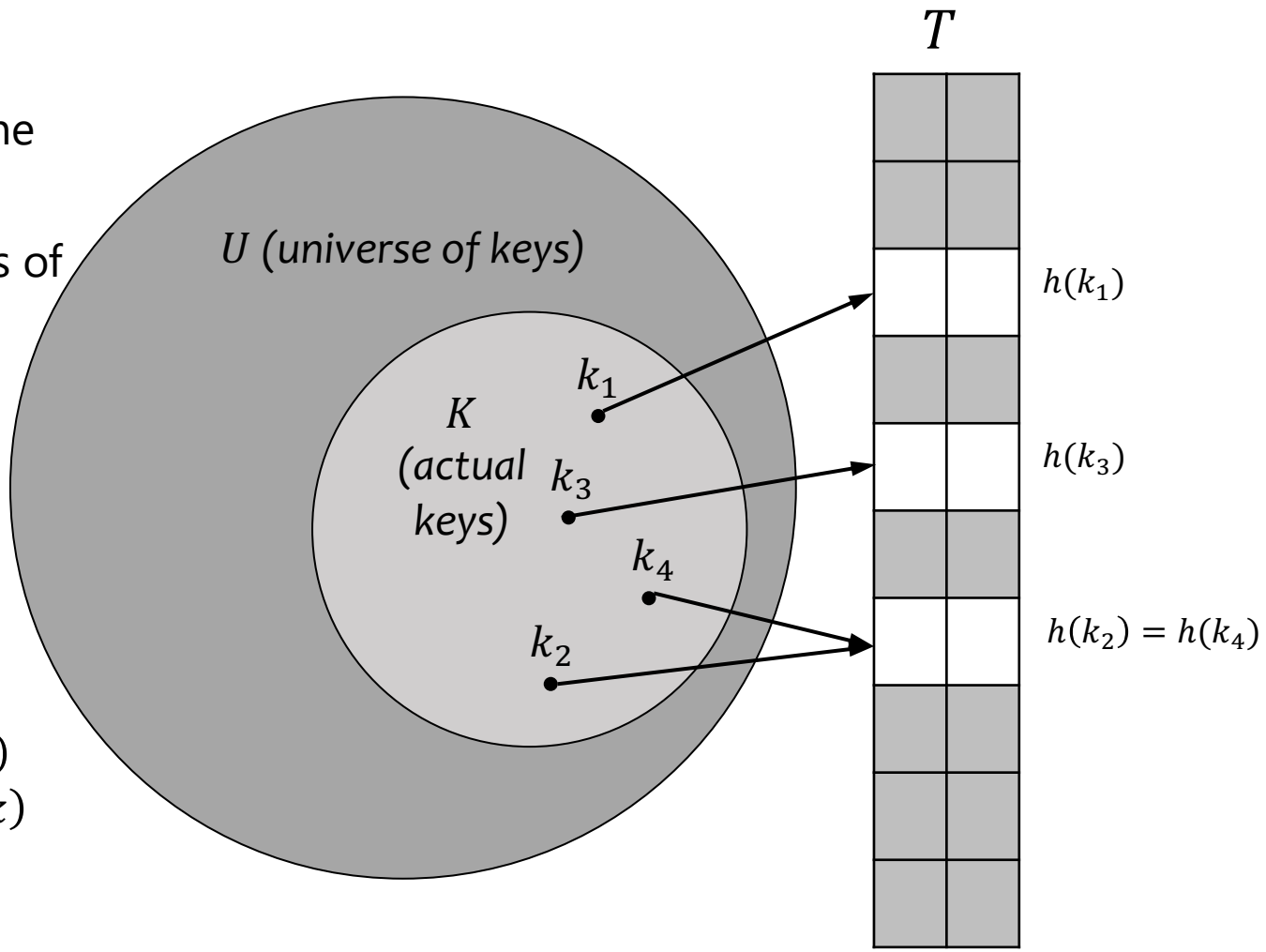
Data structure that stores the keys and values by mapping keys to their associated values via a hash function

Hash Tables

- A list of (key, value) pairs
  - Hash table associates keys with values using a hash function
- Also known as:
  - hash maps, maps, dictionaries, or associative arrays

Hash Tables

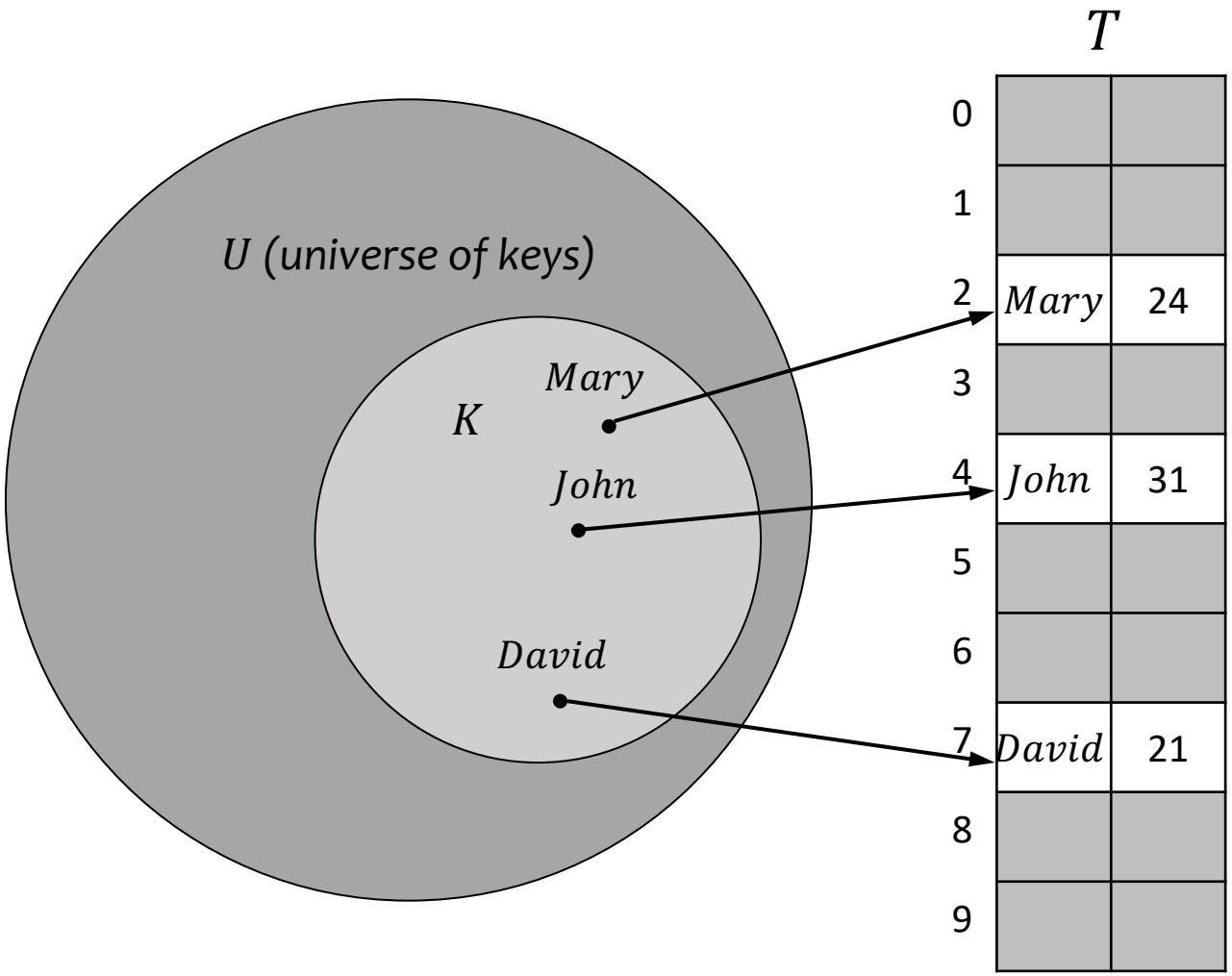
- **Hash function** h computes the slot from the key k.
- h maps the universe U of keys into the slots of a **hash table** $T[0..m-1]$
  - $h: U \to \{0, 1, \ldots, m-1\}$
  - m = the size of hash table



- An element with key k is stored in **slot** $h(k)$
  = An element with key k **hashes** to slot $h(k)$
  = $h(k)$ is the **hash value** of key k

Hash Tables

- $h("Mary") = 2$
- $h("John") = 4$
- $h("David") = 7$



Collision

- Hash functions are used to map each key to a different address space.
- Practically this is not possible, and collision occurs.
- Collision: the situation that two keys are mapped into the same location in the hash table.
  - Let hash function h be $h(x) = x.length$
  - $h("Tom") = 3$
  - $h("Sam") = 3$
- The worst-case performance for lookup is $O(n)$.
  - where n is the number of elements in a hash table.

How Can We Handle Collisions?

- Efficiency of a hash table depends on three factors
  1. The amount of data stored in the hash table
  2. The number of cells available in the hash table
  3. Hash function

Good Hash Functions

- Minimize collisions
  = Distribute key values evenly in the hash table
- Use all the information provided in the key
- Be easy and quick to compute
- Modulo arithmetic
  - $h(x) = x \; mod \; (tableSize)$
  - Table size is recommended to be prime

Collision Resolution

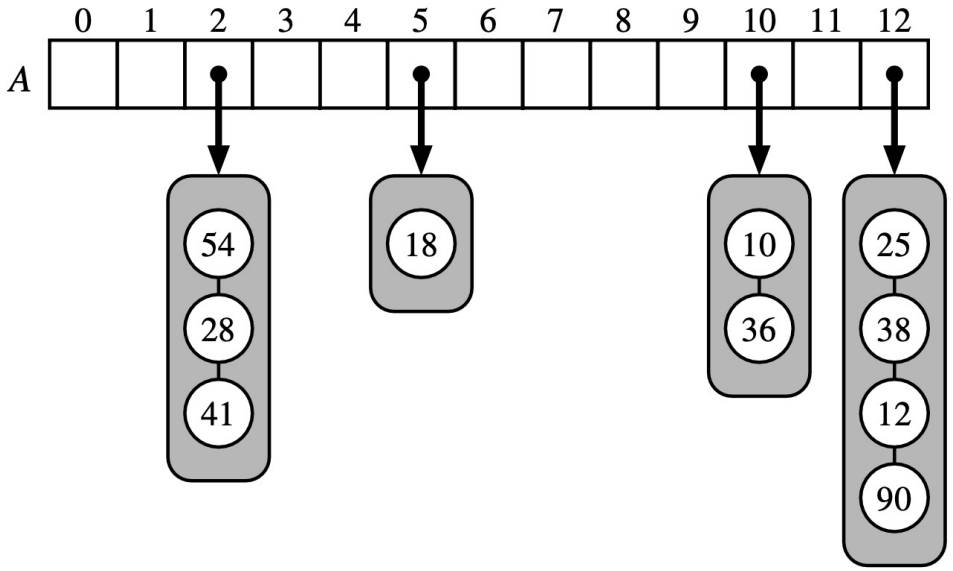
- Process of finding an alternate location when a collision occurs.
- Two techniques

1. **Separate chaining**
2. **Open addressing** (closed hashing)

```
Insert(newItem)
    if T[h(newItem)] is not occupied
        T[h(newItem)] ← newItem
    else
        Find an appropriate index x
        T[x] ← newItem
```

Separate Chaining

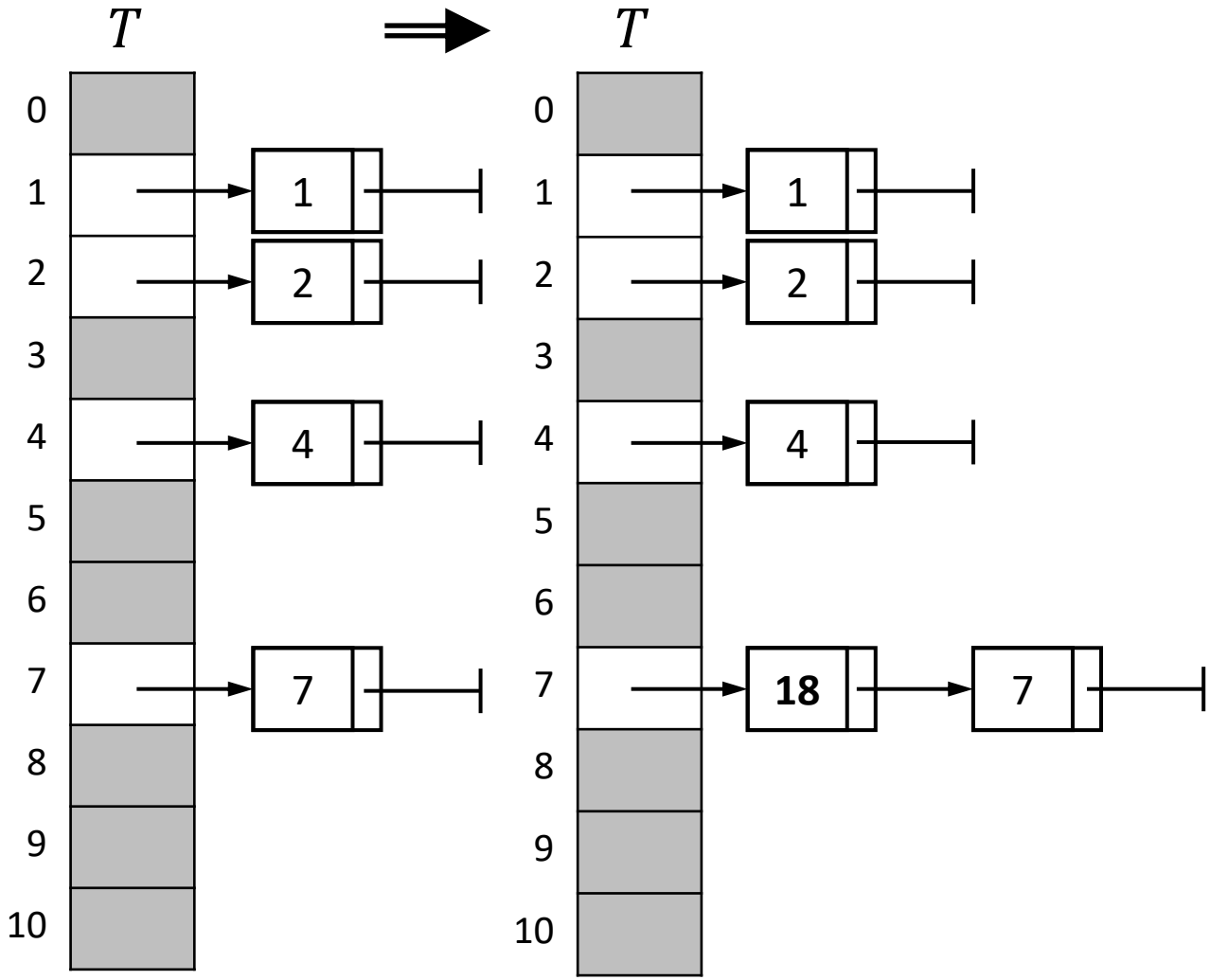
- Keep a linked list of elements that hash to the same value
- Operations
  - Find: use a hash function to determine which list to traverse
  - Insert: traverse down the list to check whether the element is in the list. If not, it is inserted at the front
- The worst-case time complexity of a hash table lookup is O(n).



Separate Chaining Example

- Suppose $h(k) = k \; mod \; 11$
- 18 is inserted to slot $T[h(18)] = T[7]$

- Need extra space and operation for pointers and new nodes



Load Factor

- The rate of occupied slots in the table

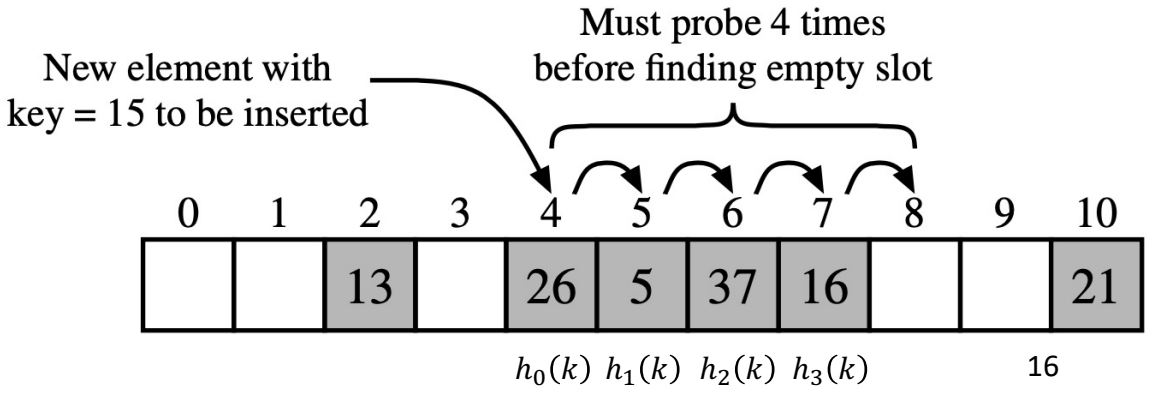
$$\lambda = \frac{n}{H}$$

- n: $Number\ of\ elements\ in\ hash\ table$, H: $Hash\ table\ size$
- λ should be bounded by a small constant, preferably below 1.
- Given a good hash function, expected runtimes in separate chaining are:
  - Unsuccessful search: $1\ + \lambda = O(1)$
    - 1 (hash function)
    - λ (the average number of elements in each bucket)
  - Successful search: 1 (hash function) $+ \frac{\lambda}{2} = O(1)$.
    - The expected length of iteration to which i-th element is the answer is $\frac{\mathrm{i}-1}{H}$
    - $\frac{1}{\mathrm{n}} \cdot \Sigma_{\mathrm{i}=1}^{\mathrm{n}}(1 + \frac{\mathrm{i}-1}{H})$

Open Addressing (Closed Hashing)

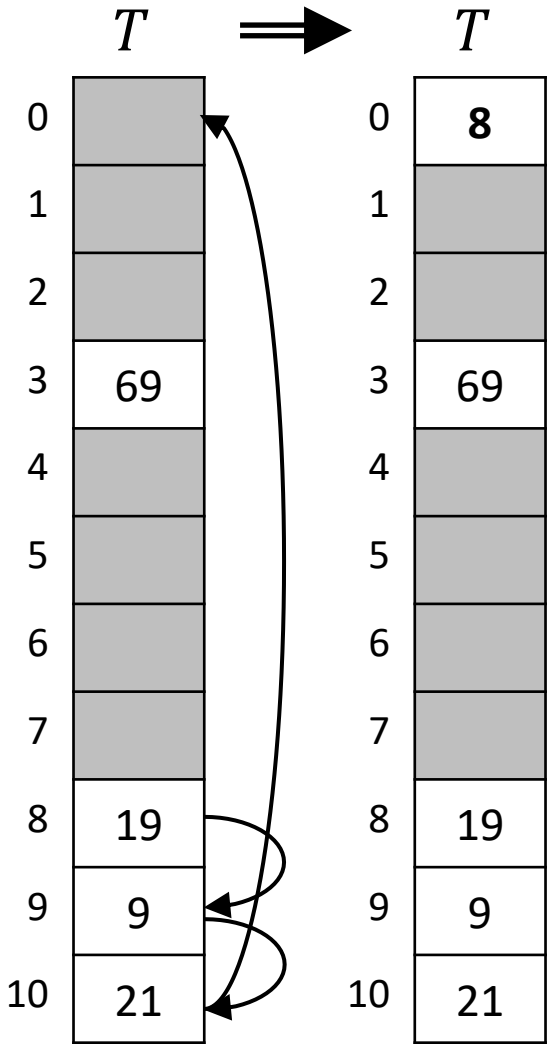
- All keys are stored in the hash table itself, but a collision is resolved by <u>probing a sequence of hash values</u> $h_0(k), h_1(k), h_2(k), \ldots$

1. **Linear probing**: linear search
   - $h(k) = k \; mod \; (tableSize)$
   - $h_i(k) = (h(k) + i) \; mod \; (tableSize)$ where $i = 0, 1, 2, \ldots$
2. **Quadratic probing**: nonlinear search
   - $h(k) = k \; mod \; (tableSize)$
   - $h_i(k) = (h(k) + i^2) \; mod \; (tableSize)$ where $i = 0, 1, 2, \ldots$
3. **Double hashing**: use two hash functions
   - $h_i(k) = \big(f(k) + ig(k)\big) \; mod \; (tableSize)$
   - $f(k), g(k)$: hash functions



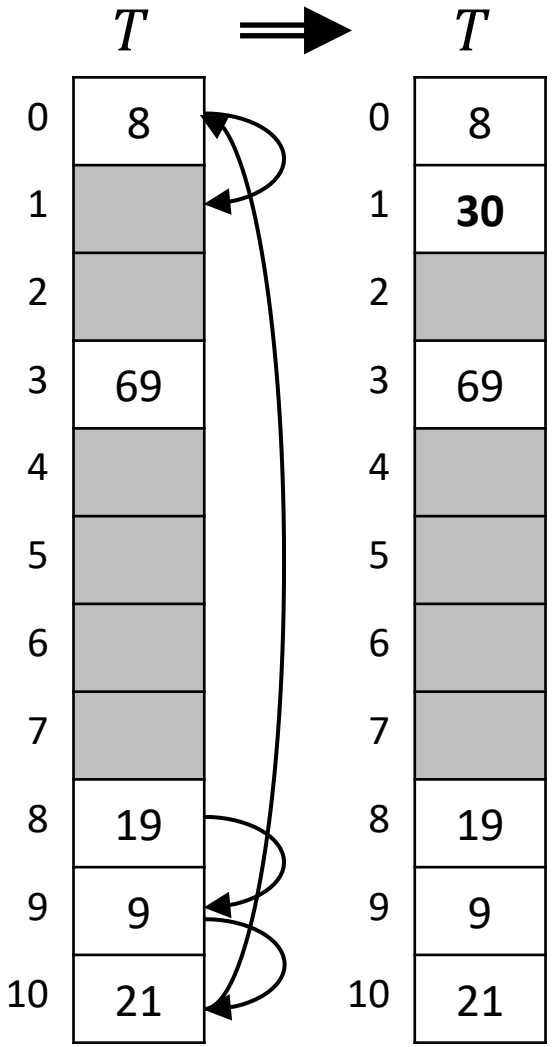
Linear Probing Example

- Given $h_i(k) = (h(k) + i)\ mod\ 11$
- Insert 8
  - Probe a sequence $h_0(8), h_1(8), h_2(8), h_3(8)$



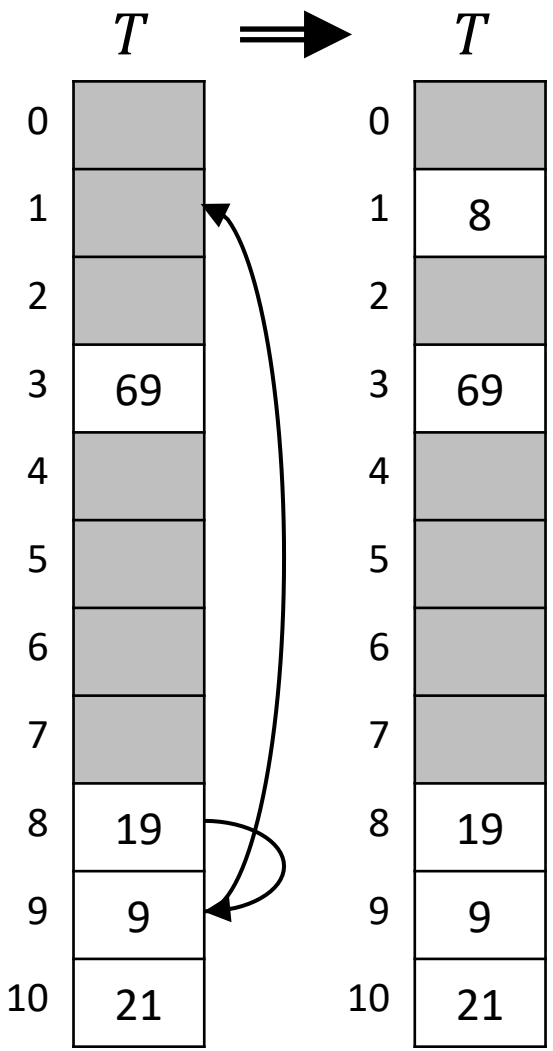
Linear Probing Example

- Given $h_i(k) = (h(k) + i)\ mod\ 11$
- Insert 30
  - Probe a sequence $h_0(30), h_1(30), h_2(30), h_3(30), h_4(30)$



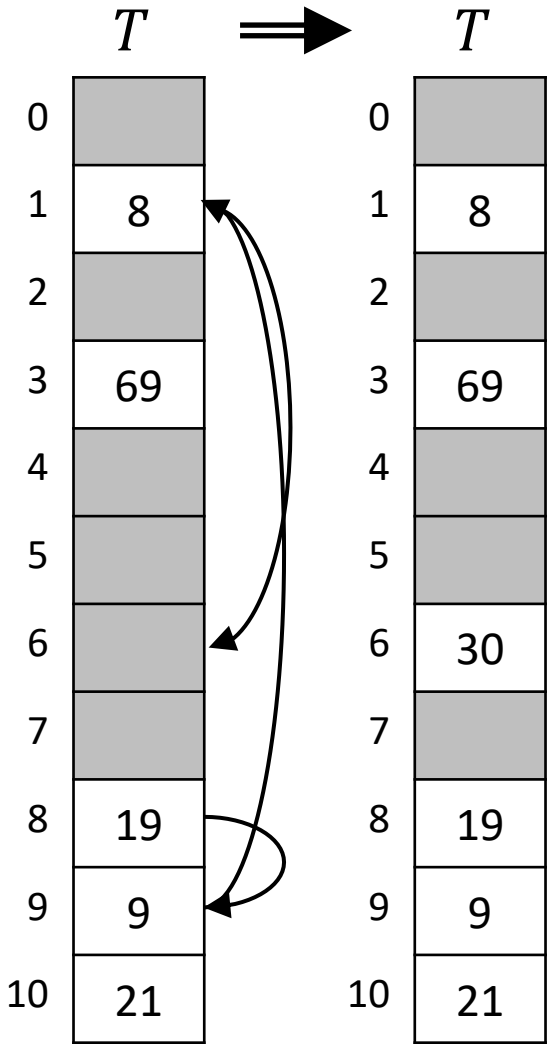
Quadratic Probing Example

- Given $h_i(k) = (h(k) + i^2)\ mod\ 11$
- Insert 8
  - Probe a sequence $h_0(8), h_1(8), h_2(8)$



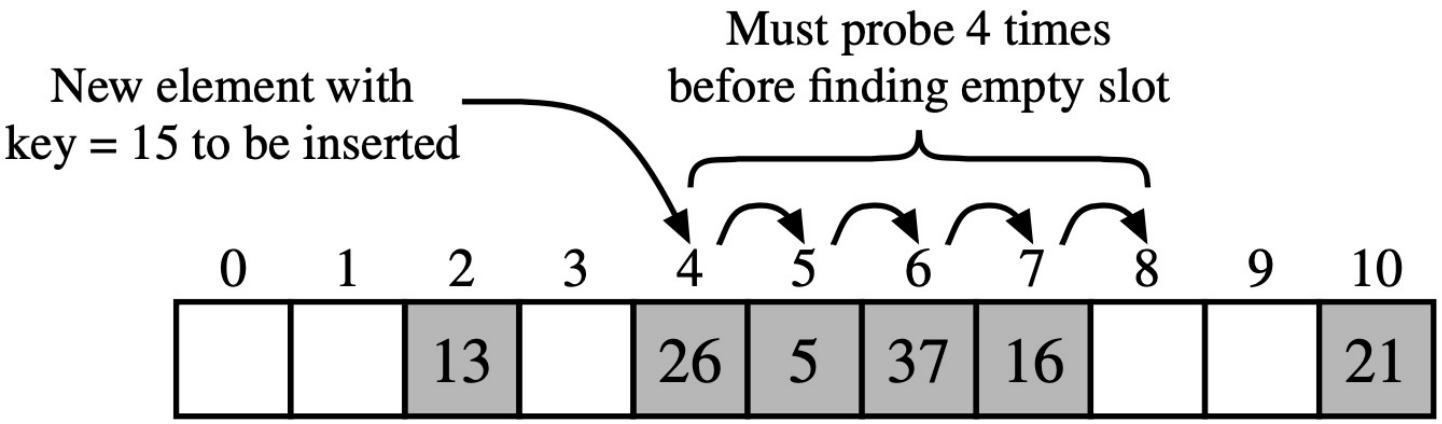
Quadratic Probing Example

- Given $h_i(k) = (h(k) + i^2)\ mod\ 11$
- Insert 30
  - Probe a sequence $h_0(30), h_1(30), h_2(30), h_3(30)$



Problems

- Primary clustering: long runs of occupied slots build up, increasing the average search time.
- Secondary clustering: the initial probe determines the entire sequence (and so only m distinct probe sequences are used).
  - m is the size of hash table

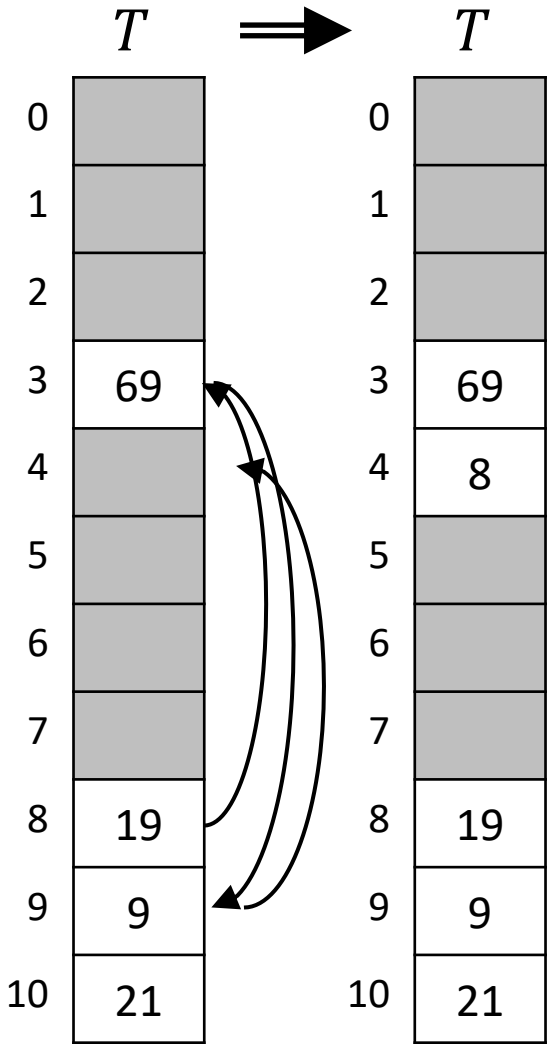


Double Hashing Example

Given:

- $f(k) = k \; mod \; 11$
- $g(k) = 7 - (k \; mod \; 7)$
- $h_i(k) = \big(f(k) + ig(k)\big) \; mod \; 11$

- Insert 8
  - Probe a sequence $h_0(8), h_1(8), h_2(8), h_3(8)$



Comparison of Open Addressing Methods

	Linear Probing	Quadratic Probing	Double Hashing
Interval between probes	Fixed (or 1)	Increases proportional to the hash value	Computed by another hash function
Efficiency	- Fast (only 1 hash function)	- Fast (only 1 hash function) - Does not probe all locations in the table	- More time to compute more than 1 hash functions - Use few probes
Problem	- Primary clustering - Secondary clustering	- Secondary clustering	- More complicated to implement

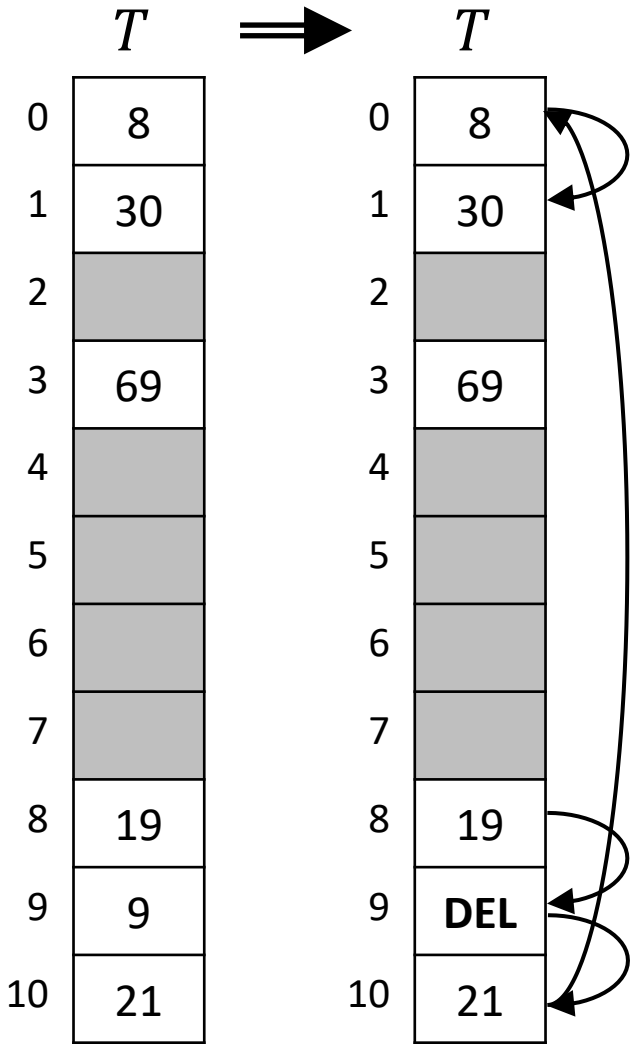
Observation

- Probing methods have little difference in performance when a load factor is small.
- Successful search (searching for the item that exists in the table) has the exactly same track as the insertion of the item.

Deletion in Closed Hashing

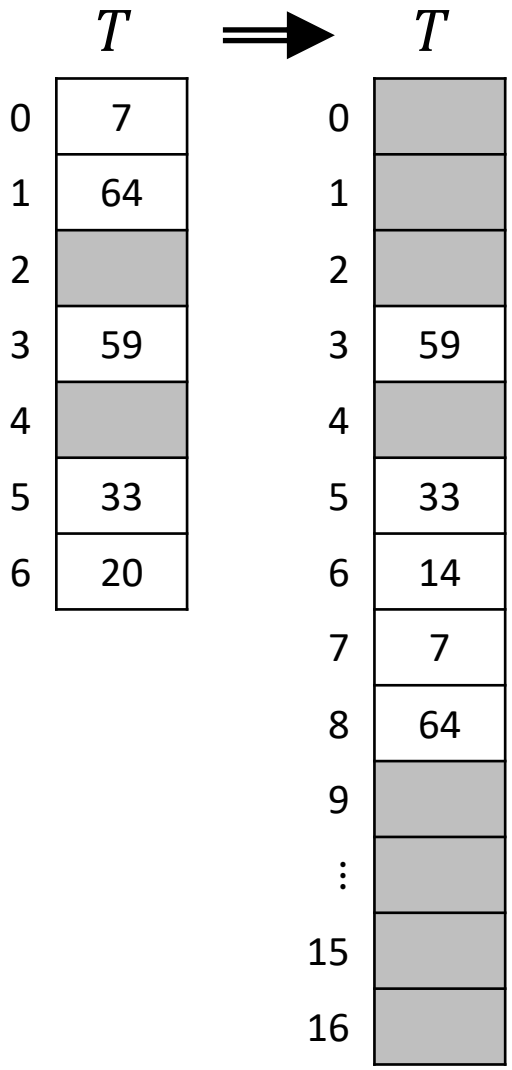
- Use special value (e.g., DEL) to distinguish deleted locations and empty locations.
  - Delete(9), and then
  - Find(30)

- If DEL is found during probing:
  - Find(x): keep searching for x until empty
  - Insert(x): reuse the DEL location for placing x



Rehashing

- A high load factor diminishes the performance.
- When the table gets nearly full, the operations take the long running time
- Build another table that is about twice as big, and rehash all the items in the new table.
- Example
  - Before rehashing: $h(k) = k \; mod \; 7$
  - After rehashing: $h(k) = k \; mod \; 17$



Time Complexity

- O(1) time to lookup, insert, and delete on average.

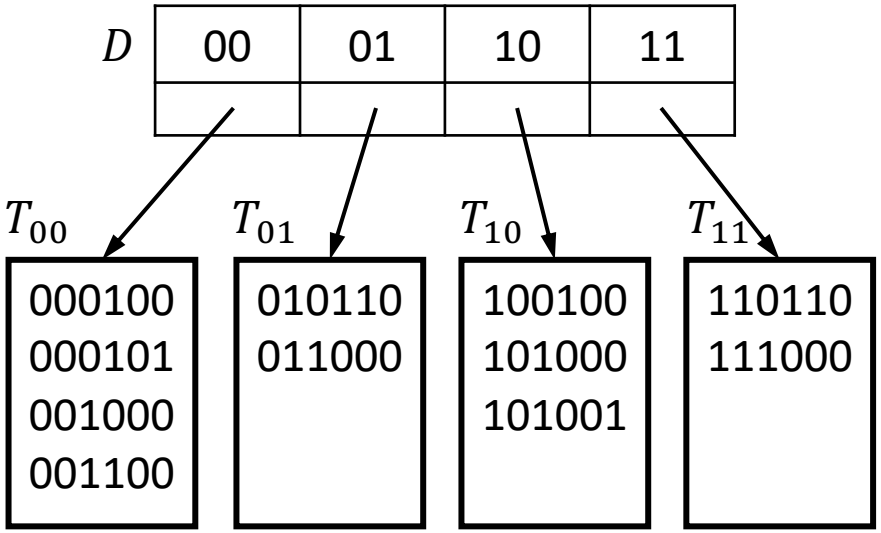
Operation	Average case	Worst Case
Lookup	$O(1)$	$O(n)$
Insertion	$O(1)$	$O(n)$
Deletion	$O(1)$	$O(n)$

Extendible Hashing

- What if the hash table is too large to fit in main memory?
- Since disk access is costly but memory access is cheap,
  - Locality matters for large data structures.
  - Need a method to reduce the number of disk access.

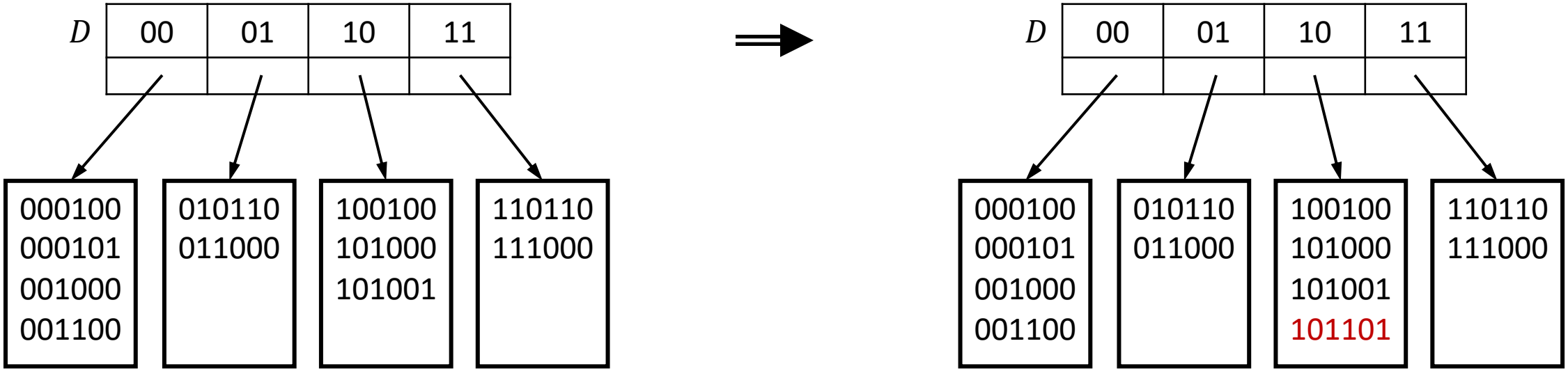
Extendible Hashing

- A hash table is broken into multiple smaller hash tables, each of which is called a **bucket**.
- Bucket
  - The maximum size of a bucket is the size of a disk page.
  - Can hold as many records that can be fit in one page as possible
  - Keeps each bucket at least half full
- To find which bucket to search for
  - Store a data structure called **directory** in main memory
  - Each entry in the directory holds a **disk address** of the corresponding bucket.



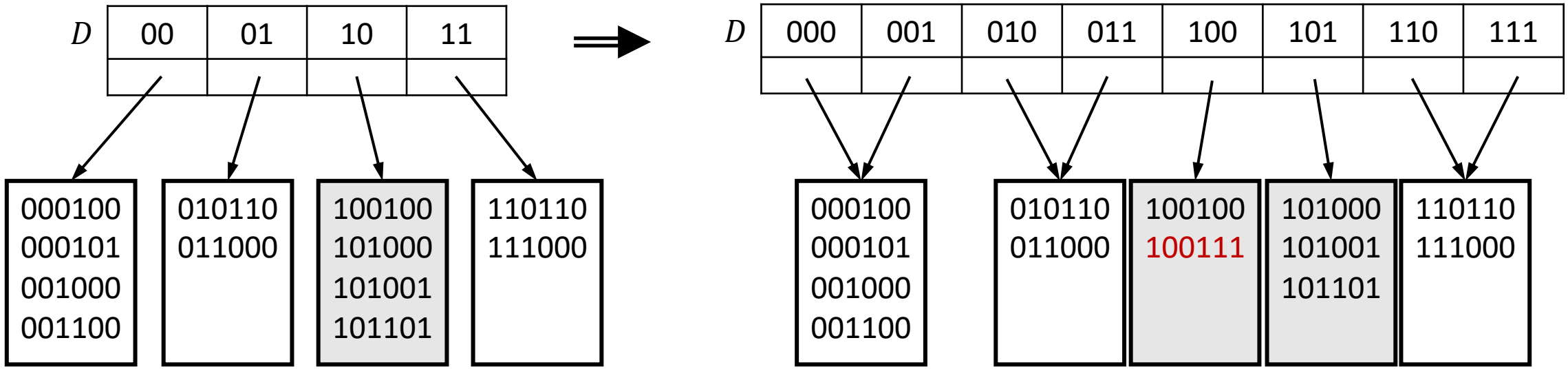
Extendible Hashing Example

- Insert 101101



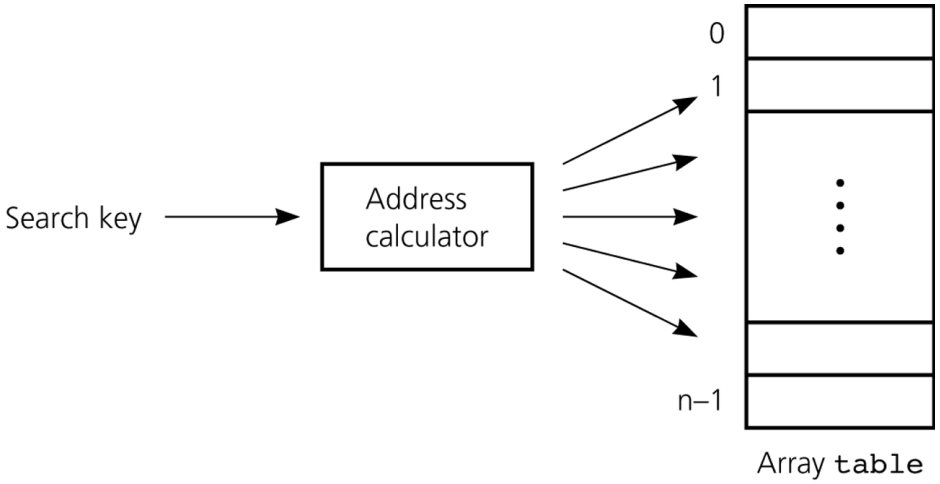
Extendible Hashing Example

- Insert 100111



Takeaways

- Stacks, queues, priority queues
  - Do not support the search operation = Do not support dictionary
- Hash tables
  - Do not support finding the minimum (or maximum) element
- Applications that need radically fast operations
  - 119 Emergent calls and locating caller's address
  - Air flight information system
  - Residence registration management system



Exercise 1

- Given an array $A[0..n-1]$ of n numbers containing the repetition of some number.
- Give an algorithm for checking whether there are repeated elements or not.
- Example 1
  - Input: [5, 7, 1]
  - Output: False
- Example 2
  - Input: [5, 7, 1, 5]
  - Output: True

Exercise 2

- Given two arrays, determine whether one array is a subset of another array.
- Example 1
  - Input: a=[4, 8], b=[2, 4, 5, 8]
  - Output: True
- Example 2
  - Input: a=[4, 6, 8], b=[2, 4, 5, 8]
  - Output: False